Word Embeddings

A Complete Guide to Word Representations in NLP

The challenge: machines need vectors, not raw text

1. Introduction: Why Do We Need Word Embeddings?

When you read the sentence "I saw a cat," your brain instantly understands the meaning — the object, its properties, the context. But a machine learning model cannot work with raw text. It needs numbers. Specifically, it needs vectors of features that capture the meaning of words in a mathematical form.

> **Analogy: Words are like people's ID cards**
>
> *Just as a passport turns a person's identity into a set of numbers and fields (age, nationality, height), word embeddings turn a word's "identity" — its meaning and usage — into a set of numbers (a vector). The better the embedding, the richer the ID card.*

These numerical representations of words are called word embeddings (or word vectors). They are the foundational building blocks that let NLP models understand language.

> **Key Concept: What is a Word Embedding?**
>
> A word embedding is a dense vector of real numbers that represents a word. Unlike a simple index, this vector encodes semantic meaning — words with similar meanings should have similar vectors. For example, the vectors for "cat" and "dog" should be closer to each other than either is to "table".

1.1 The Look-up Table

In practice, the system works as follows: you define a fixed vocabulary of allowed words in advance. For each word in that vocabulary, a look-up table stores its embedding vector. To retrieve the embedding for a word, you simply look it up using the word's index in the vocabulary.

A look-up table: each word index maps to its dense vector representation

What happens with words not in the vocabulary (called out-of-vocabulary or OOV words)? There are three common approaches:

- Assign them to a special UNK (unknown) token that has its own vector
- Ignore them completely
- Assign them a zero vector

> **The Central Question of this Guide**
>
> The look-up table is just a mechanism for storage and retrieval. But the real question is: HOW do we get good word vectors in the first place? That is exactly what this entire guide is about.

2. One-Hot Vectors: The Naive Approach

The simplest possible approach is called one-hot encoding. For a vocabulary of size V, the vector for the i-th word has a 1 in the i-th position and 0 everywhere else. This is the most basic way to represent categorical features in machine learning.

One-hot vectors: for a vocabulary of size V, each word gets a vector with exactly one "1"

Analogy: One-hot vectors are like lottery tickets with only one number

Imagine describing every person using a unique lottery ticket number. Person #342 is described only as "342." You cannot tell from the numbers alone that #342 and #341 are siblings, or that #342 and #500 both love pizza. The numbers carry no relational meaning — they're just labels.

Why One-Hot Vectors Are Not Enough

There are two major problems with one-hot vectors:

#	Problem	Details
1	**Enormous Size**	For a vocabulary of 50,000 words, every word gets a 50,000-dimensional vector with 49,999 zeros. Extremely wasteful.
2	**No Semantic Meaning**	One-hot vectors think "cat" is equally distant from "dog" and from "table"! They cannot capture that cat and dog are both animals.

Problem #2 is the critical one. One-hot vectors are essentially random labels. They tell us nothing about what words mean or how they relate to each other. This leads us to the central question: how do we define and capture "meaning"?

3. Distributional Semantics: Meaning from Context

Before we can build better word vectors, we need a working definition of "meaning" that can be used computationally. The key insight comes from linguistics.

The Tezgüino Thought Experiment

Imagine you encounter a completely unknown word: "tezgüino." You see it used in sentences like:

- "A bottle of tezgüino is on the table."
- "Everyone likes tezgüino."
- "We make tezgüino out of corn."
- "Tezgüino makes you drunk."

From these contexts alone, you can deduce that tezgüino is probably an alcoholic beverage, something like beer or wine. You recognized its meaning by recognizing that it appears in the same contexts as known drinks!

Analogy: You are a new employee at a company

Imagine starting a new job and not knowing who "Alex" is. But you hear: "Alex approved the budget," "Alex runs the engineering team," "Alex has final say on hiring." You quickly deduce Alex is a senior manager — just from the contexts Alex appears in. That's distributional semantics at work.

Ő Key Concept: The Distributional Hypothesis

"Words which frequently appear in similar contexts have similar meaning." — This idea is often summarized as: "You shall know a word by the company it keeps" (J. R. Firth, 1957). The practical implication: to capture meaning, we need to capture context.

This insight is enormously valuable because it gives us something we can compute. Instead of needing a linguist to define meaning, we just need to count how words co-occur in text. The distributional hypothesis tells us:

"Capturing meaning"	"Capturing contexts"
...is the same as...	So we just need to build context-rich representations!

4. Count-Based Methods

Count-based methods take the distributional hypothesis very literally: they build word vectors by directly counting how often words co-occur in text. The general approach has two steps:

1. Build a word-context co-occurrence matrix from a large corpus
2. Reduce the dimensionality of that matrix (for efficiency and quality)

The two-step pipeline: build a co-occurrence matrix, then reduce its dimensionality

Why reduce dimensionality? Two reasons: (1) the raw matrix is enormous, and (2) many words appear in only a few contexts, leaving the matrix sparse with many zeros that carry no information.

Analogy: Count-based methods are like a city's commute map

Imagine a map showing how many people travel between every pair of locations in a city each day. That's the raw co-occurrence matrix. But it's too complex to analyze directly. Dimensionality reduction is like finding the key "hubs" — the subway stations and major roads — that explain most of the traffic patterns. The reduced representation is smaller but captures the essential structure.

4.1 Simple Co-Occurrence Counts

The simplest approach: define a context window of size L around each word. Count how many times each word appears in the context window of each other word. Matrix entry (w, c) = number of times word w appears in context of word c.

For example, in the sentence "The cute grey cat was playing in the garden," with a window of size 2 around "cat," the context words are: cute, grey, was, playing. Each of these gets a co-occurrence count of 1 with "cat."

The HAL Model (1996)

The Hyperspace Analogue to Language model was one of the first famous applications of this approach. It also added a positional weighting — words closer to the center word get higher weight — and tracked left vs. right context separately.

4.2 Positive Pointwise Mutual Information (PPMI)

Simple counts have a problem: very frequent words (like "the," "is," "a") co-occur with almost everything, making their co-occurrence counts uninformative. Pointwise Mutual Information (PMI) is a smarter measure that asks: does word w appear with context c more often than you would expect by chance?

$$PMI(w, c) = \log [P(w, c) / (P(w) \times P(c))]$$

If word w and context c co-occur more than chance, PMI > 0. If less than chance, PMI < 0.

PPMI: using mutual information instead of raw counts for a smarter measure of word-context association

PPMI (Positive PMI) takes only the positive values of PMI, setting negative values to 0. Negative PMI values are unreliable (they would require enormous corpora to be accurate), so PPMI discards them. PPMI is widely considered the state-of-the-art pre-neural method for distributional similarity.

> **Analogy: PPMI is like a friendship surprise score**
>
> *Suppose 10% of people are doctors and 10% go to jazz clubs. By chance alone, 1% of people are both. If you see that 5% of doctors attend jazz clubs — 5 times more than chance — that's a strong association (high PMI). PPMI only records surprisingly strong connections, not boring coincidences.*

4.3 Latent Semantic Analysis (LSA)

LSA takes a different angle: instead of word-word co-occurrence, it builds a term-document matrix. Each row is a word, each column is a document, and entries can be simple counts or TF-IDF scores. Then it applies Singular Value Decomposition (SVD) to reduce dimensionality.

LSA: analyzing word-document co-occurrence to find latent topics and produce both word and document vectors

What makes LSA special is that it produces vectors for both words AND documents. The cosine similarity between document vectors can be used to measure how similar two documents are in topic. This makes LSA one of the simplest topic models.

> **Analogy: LSA discovers hidden "themes" like a book club analyst**
>
> *Imagine you have thousands of book reviews. LSA is like an analyst who reads all reviews and discovers hidden themes: "mystery-thriller," "romance," "sci-fi." Once the themes are found, every book can be described as a mix of these themes (a vector), and similar books cluster together — even if they never share the exact same words.*

5. Word2Vec: Learning to Predict Context

While count-based methods directly encode context statistics into vectors, Word2Vec takes a completely different approach: it learns word vectors by training a model to predict context words. The vectors emerge as a byproduct of this prediction task.

Ŏ Key Concept: Word2Vec's Core Philosophy

Instead of manually computing context statistics (like count-based methods), Word2Vec trains word vectors to predict their context. The better a word vector is at predicting its surrounding words, the more it must have "understood" the word's meaning and usage.

Word2Vec overview: word vectors are parameters optimized to predict context words

Analogy: Word2Vec is like studying for a fill-in-the-blank exam

Imagine your teacher gives you millions of sentences with one word blanked out. You study until you can consistently predict the missing word from context. After this training, your internal model of each word must be rich enough to place it correctly in context. That's exactly what Word2Vec does — the "exam studying" is gradient descent, and your internal model is the word vectors.

5.1 The Sliding Window Algorithm

Word2Vec's training process works as follows:

1. Take a huge text corpus (billions of words)
2. Slide a window of size m over the text, one word at a time
3. At each position, you have a central word and context words (all other words within the window)
4. Compute the probability that each context word appears given the central word
5. Adjust the vectors to increase these probabilities

The sliding window: at each step, one central word and surrounding context words

5.2 The Objective Function: Negative Log-Likelihood

Formally, for each position t in the corpus, Word2Vec predicts all context words within an m-sized window given the central word. The objective is to maximize the likelihood of all context words — equivalently, to minimize the average negative log-likelihood (the loss function):

The loss function: negative log-likelihood over all word-context pairs in the corpus

This loss function elegantly captures the training procedure: for every word in every position in the text, we want the model to assign high probability to all words in its context window.

> **Analogy: The loss function is a student's report card**
>
> *Imagine the loss as a penalty score you get wrong answers on. For every sentence in every book you've ever read, you're asked: "Given the word 'cat', what words are likely nearby?" If you predict 'dog' and 'cute' but miss 'meow' and 'fur', you get penalized. Training means practicing until your report card (the loss) is as low as possible.*

5.3 Computing Probabilities: Two Vectors Per Word

For each word w, Word2Vec maintains TWO vectors:

- v_w — the "central word" vector, used when w is the focus word
- u_w — the "context word" vector, used when w appears in someone else's context

After training, context vectors are discarded and only the central-word vectors are used. The probability of a context word o given central word c is computed using the softmax function:

The softmax probability: high dot product between central and context vectors → high probability

```
P(o | c) = exp(u_o · v_c) / Σ_w exp(u_w · v_c)
```

This is the softmax function. It converts dot products into a valid probability distribution.

Why two vectors? It's a clever mathematical trick: keeping central and context vectors separate makes the loss function linear in the parameters, which makes gradients easy to compute.

> **What is Softmax?**
>
> Softmax is a mathematical function that takes any set of numbers and converts them into a probability distribution (all positive, sum to 1). The "max" part means the highest input gets the highest probability; the "soft" part means all probabilities remain non-zero. You'll encounter softmax constantly in deep learning.

5.4 Training: Gradient Descent

Training happens through gradient descent: we compute the gradient of the loss with respect to the word vectors and update them in the direction that decreases the loss.

$$\theta_new = \theta_old - \alpha \cdot \nabla_\theta J(\theta)$$

α is the learning rate; θ represents all word vectors (both central and context vectors)

Updates happen one word-context pair at a time. For the central word "cat" with context word "cute," the update:

- Increases the similarity (dot product) between v_cat and u_cute
- Decreases the similarity between v_cat and u_w for all other words w

One step of the Word2Vec training algorithm: update vectors to increase probability of the true context word

You might worry: why decrease similarity with all other words, even ones that are also valid contexts (like "grey" or "playing")? The answer: since we repeat this process for every word in every sentence, averaged over the entire corpus, the vectors converge to capture the full distribution of contexts.

5.5 Faster Training: Negative Sampling

The standard softmax has a major efficiency problem: for every single training step, you must compute dot products with ALL words in the vocabulary (potentially 50,000+ words) to normalize the probability. This is extremely slow.

Negative Sampling is the solution: instead of comparing against all vocabulary words, we compare against only K randomly chosen "negative" (non-context) words. This reduces the work per step from O(|V|) to O(K), where K is typically just 2–20.

Negative sampling: instead of all vocab words, only use a small set of random "negative" examples

$$J(\theta) = -\log \sigma(u_cute \cdot v_cat) - \Sigma_k \log \sigma(-u_k \cdot v_cat)$$

Maximize similarity with the true context word, minimize similarity with K random negatives

Analogy: Negative sampling is like studying with practice tests

Instead of memorizing the entire dictionary for every vocabulary question, you study by answering: "Is THIS specific word a good context for 'cat'?" where you see the correct word plus a few random imposters. You don't need to consider all 50,000 words — just a few key examples each time. Over millions of examples, you still learn the full picture.

Choosing Negative Examples

How are the K negative words chosen? Word2Vec samples them from a modified unigram distribution — words are sampled proportional to their frequency raised to the power 3/4:

$$P(w) \propto U(w)^{(3/4)}$$

U(w) is the raw frequency of word w; raising to 3/4 boosts rare words relative to their natural frequency

Why the 3/4 power? Pure frequency-based sampling would overwhelmingly pick common words like "the" and "is" as negatives, which are not very informative. The 3/4 power smooths the distribution, giving rare words a better chance of being selected as negatives.

5.6 Skip-Gram vs CBOW

Word2Vec comes in two variants:

Skip-Gram (left): predict context from center word. CBOW (right): predict center word from context sum

Skip-Gram	CBOW
• Predicts context words given the central word • More popular variant (especially SGNS) • Better for infrequent words • Produces higher-quality representations overall • Slower to train than CBOW	• Predicts the central word from sum of context vectors • "Continuous Bag of Words" — ignores word order • Faster to train • Works better for frequent words • The "bag of words" sum loses positional information

5.7 Additional Notes on Word2Vec

Window Size Effects

The size of the context window strongly affects the resulting vectors:

- Larger windows (5–10 words): vectors capture topical similarity. Words like "dog," "bark," and "leash" cluster together because they all appear in articles about dogs, even if they don't appear next to each other.
- Smaller windows (1–2 words): vectors capture functional/syntactic similarity. Words like "walking," "running," and "approaching" cluster together because they appear in the same grammatical positions.

Standard Hyperparameters

A commonly used configuration:

- Model: Skip-Gram with negative sampling (SGNS)
- Number of negative examples: 2–5 for large corpora, 15–20 for smaller ones
- Embedding dimensionality: 300 (common default; 50 or 100 also work well)
- Context window size: 5–10 words

Frequency Subsampling

To handle the fact that very frequent words (like "the," "a," "in") carry less information, Word2Vec subsamples them during training. Each word is randomly ignored with probability:

$$P(w_i) = 1 - \sqrt{(thr / f(w_i))}$$

f(w_i) is the word frequency; thr is a threshold (typically 10^{-5}). Frequent words get dropped more often.

This trick both speeds up training AND improves the quality of rare word vectors, because the model spends less time on uninformative frequent words.

Relation to PMI Matrix Factorization

v Deep Connection: Word2Vec ≈ PMI Matrix Factorization

A landmark 2014 paper by Levy & Goldberg showed that Word2Vec (Skip-Gram with Negative Sampling) is implicitly factorizing a PMI matrix shifted by log(k), where k is the number of negative samples. This bridges the gap between "neural" and "count-based" methods — they are mathematically more similar than they appear!

6. GloVe: Global Vectors for Word Representation

GloVe (Global Vectors for Word Representation) is a third approach that bridges count-based and prediction-based methods. The name "Global Vectors" reflects the key insight: it uses global corpus statistics, not just local sliding windows.

GloVe combines global co-occurrence statistics with a learned objective function

Ő Key Concept: GloVe's Core Idea

GloVe builds on the simple co-occurrence count matrix N(w, c) — the number of times word w and context c appear together. But instead of just using these counts directly (like LSA), it defines a loss function that trains vectors to predict these counts. This gives it the benefits of both worlds: the efficiency of global statistics and the quality of learned representations.

6.1 The GloVe Objective

The GloVe loss function: trains vectors to predict log co-occurrence counts, with clever weighting

The loss function has a key feature: weighting. GloVe controls the influence of rare vs. frequent word pairs:

- Rare co-occurrence pairs: penalized (treated as less reliable data)
- Very frequent co-occurrence pairs: not over-weighted (to avoid frequency dominating everything)

Analogy: GloVe's weighting is like a pollster weighting survey responses

A good pollster doesn't just count all votes equally. Someone who barely knows a candidate's name gets less weight than someone who attended three of their rallies. But a hardcore supporter's voice also shouldn't dominate over thousands of moderate respondents. GloVe's weighting function balances rare (unreliable) and very frequent (over-dominant) observations.

6.2 GloVe vs Word2Vec

GloVe	Word2Vec
• Uses global co-occurrence statistics • Batch-style training on the co-occurrence matrix • Averages word + context vectors after training • Conceptually: log-bilinear regression on co-occurrence counts • Good at capturing global statistical patterns	• Uses local context windows (sliding) • Online training on individual word-context pairs • Discards context vectors after training • Conceptually: implicit PMI matrix factorization • With tuned hyperparameters, generally outperforms GloVe

A Surprising Research Finding

Many early papers claimed Word2Vec (prediction-based) was superior to count-based methods, and that GloVe was better than Word2Vec. A 2015 paper by Levy, Goldberg, and Dagan showed: when hyperparameters are properly tuned, count-based PPMI methods are competitive with Word2Vec — and Word2Vec (SGNS) consistently outperforms GloVe. The apparent differences were largely due to implementation choices, not fundamental algorithm differences.

7. Evaluating Word Embeddings

How do you know if one set of word embeddings is better than another? There are two fundamentally different evaluation paradigms, each with its own advantages and disadvantages.

7.1 Intrinsic Evaluation: How Good are the Vectors Themselves?

Intrinsic evaluation measures the quality of the embeddings directly, without applying them to a specific downstream task. It asks: do the vectors capture human intuitions about word relationships?

Intrinsic evaluation: testing word similarity and analogy tasks that assess embedding quality directly

Word Similarity Benchmarks

These test sets contain word pairs with human-annotated similarity scores (e.g., "cat" and "dog" have similarity 8.5/10, "cat" and "car" have 1.2/10). The quality of embeddings is measured as the correlation between the model's cosine similarities and human judgments. Popular benchmarks include WordSim-353 and SimLex-999.

7.2 Extrinsic Evaluation: Performance on Real Tasks

Extrinsic evaluation tests embeddings on real downstream NLP tasks: text classification, named entity recognition, machine translation, etc. You train a model with each set of embeddings and compare performance.

Extrinsic evaluation: comparing embedding quality by measuring final task performance

7.3 The Evaluation Trade-off

The trade-off: intrinsic evaluation is fast and cheap, extrinsic is slow but more meaningful

Intrinsic Evaluation	Extrinsic Evaluation
✓ Fast and cheap to compute	✓ Directly measures what you actually care about
✓ Provides quick feedback during development	✓ Captures embedding quality for YOUR specific use case
✓ No need to train task-specific models	✗ Requires training full task models (slow and expensive)
✗ May not correlate with real-task performance	
✗ Benchmark quality depends on human	

annotators	✗ Must be repeated for each new embedding set ✗ Results are task-specific, may not generalize

In practice: start with intrinsic evaluation for quick iteration, then use extrinsic evaluation before making final decisions — especially if training resources allow it.

8. Analysis & Interpretability of Word Embeddings

One of the most exciting aspects of word embeddings is what they learn on their own — properties that were never explicitly taught, but emerge naturally from the training process. This section explores what's "inside" well-trained word vectors.

8.1 Nearest Neighbors: Semantic Clustering

The simplest way to inspect embeddings: for a given word, find its nearest neighbors in vector space (measured by cosine similarity or Euclidean distance). If the embeddings are good, neighbors should be semantically related.

Even rare, technical words like "leptodactylidae" (a frog family) cluster correctly near "frog"

What's remarkable: embeddings generalize beyond common words. Scientific Latin terms, obscure place names, and specialized jargon end up in the right neighborhoods — not because the model was told what they mean, but because of the contexts they appear in.

Word similarity benchmarks measure how well cosine similarity aligns with human similarity judgments

8.2 Linear Structure: The "King − Man + Woman = Queen" Property

The most famous and surprising property of word embeddings: many semantic and syntactic relationships are (approximately) linear in vector space. This means you can do arithmetic with meaning.

The famous analogy: king − man + woman ≈ queen. The gender direction is nearly the same for royals and commoners.

Ŏ Key Concept: Linear Semantic Relationships

If you subtract the vector for "man" from "king" and add "woman," you get a vector very close to "queen." This works because the difference between "man" and "woman" (the gender direction) is nearly the same as the difference between "king" and "queen." The geometry of the space encodes semantic relationships.

Linear structure holds for many relations: country-capital, verb tenses, plural forms

ऽ Analogy: Word vectors are like a GPS coordinate system for meaning

In GPS coordinates, "north" is always a consistent direction regardless of where you are. Similarly, in word vector space, "more formal" or "female version of" are consistent directions. Moving "north" from any city takes you to cities that are north of their respective starting points. Moving in the "female" direction from any male-gendered word gives you the female equivalent.

Word Analogy Benchmarks

This linear structure inspired analogy evaluation: given the relationship (man, woman) and the word "king," can the model find "queen"? Formally:

```
find w* = argmax cosine_similarity(v_w, v_king - v_man +
                                   v_woman)
```

The answer "queen" should be the nearest vector to this algebraic combination

The word analogy task: "a is to b as c is to ___?" answered by vector arithmetic

8.3 Cross-Lingual Similarities

Remarkably, the linear structure of word embeddings extends across languages. If you train separate embeddings for English and French, the internal geometry of the two spaces is nearly the same — the same semantic relationships appear in both spaces, just rotated differently.

Cross-lingual matching: a linear transformation W maps one language's embedding space to another

This insight led to cross-lingual embedding methods. The key approach (Mikolov et al., 2013): given a small dictionary of word pairs (e.g., the 5,000 most frequent words with translations), find a linear transformation matrix W such that W × French_vector ≈ English_vector for all dictionary pairs.

$$W = \text{argmin}\ \Sigma_i\ \|W \cdot z_i - x_i\|^2$$

x_i = English vector, z_i = French vector; W is learned by gradient descent on the dictionary pairs

Zero-Shot Cross-Lingual Transfer

Later research showed you don't even need a dictionary! By matching the statistical structure of two embedding spaces (using only monolingual text in each language), you can build a cross-lingual mapping with no parallel data at all. This opens the door to unsupervised machine translation.

9. Advanced Topics

9.1 FastText: Subword Embeddings

Standard Word2Vec and GloVe assign a single vector to each word, ignoring the word's internal structure. FastText (Bojanowski et al., 2017) extends this idea to character n-grams.

FastText: a word's vector is the sum of vectors for the word itself plus all its character n-grams

For a word like "running," FastText uses vectors for the character n-grams: <ru, run, unn, nni, nin, ing, ng>, plus the full word <running>. The word's final representation is the sum of all these vectors.

Why is this useful?

- Morphological understanding: "run," "running," "ran" share n-grams and thus share vector components
- Out-of-vocabulary words: even words not in the training vocabulary can be represented from their character n-grams
- Misspelling robustness: "runnng" (typo) still shares most n-grams with "running" and gets a similar vector

> **Analogy: FastText treats words like Scrabble tiles**
>
> *In Scrabble, your score depends on the letters you play. FastText is similar: each word's "score" (representation) is built from its constituent letter-groups (n-grams). Just as "RUNNING" and "RUNNER" share tiles, they share n-gram vectors — so similar words end up with similar representations even if you've never seen one of them before.*

9.2 Biases in Word Embeddings

Because word embeddings are trained on real-world text (written by humans with biases), they absorb and amplify social biases present in that text.

Gender bias in word2vec: "man is to programmer as woman is to homemaker" — an undesired learned association

A famous 2016 paper by Bolukbasi et al. found that Word2Vec embeddings encoded gender stereotypes:

- "Man is to computer programmer as woman is to homemaker"
- "Man is to doctor as woman is to nurse"
- Occupations like "captain," "architect," "warrior" had strongly male-coded vectors
- Occupations like "homemaker," "nurse," "librarian" had strongly female-coded vectors

Bias is a Serious Problem

When biased embeddings are used in real-world systems (e.g., resume screening, loan decisions, medical diagnosis), they can perpetuate and amplify discrimination at scale. Debiasing embeddings is an active research area. Two notable approaches: (1) Bolukbasi et al. (2016) proposed post-processing to project out the gender direction from gender-neutral words; (2) Ravfogel et al. (2020) proposed Iterative Nullspace Projection — repeatedly training a linear classifier to predict the protected attribute and then nullifying that information.

9.3 Semantic Change Detection

Word embeddings trained on different corpora (different time periods, different communities, different geographic regions) can be compared to detect words whose meaning has shifted.

The key insight: if a word is used differently in two corpora, its nearest neighbors in the embedding space will be different. Two main approaches:

Method 1: Align & Compare (ACL 2016)	Method 2: Neighbor Intersection (ACL 2020)
1. Train embeddings on each corpus separately 2. Find rotation matrix R that aligns the two spaces (Orthogonal Procrustes) 3. Words whose vectors don't match well after alignment have changed meaning	1. Train embeddings on each corpus separately 2. For each word, find the k nearest neighbors in each space 3. Small intersection = word changed meaning. Simpler and more stable than alignment!

Analogy: Semantic change detection is like tracking fashion trends

"Cool" used to mean cold temperature. Then it became slang for stylish. "Gay" used to mean happy. Then it changed meaning entirely. By training embeddings on text from different decades, you can automatically detect these shifts: "cool" from the 1950s has different nearest neighbors than "cool" from the 2020s.

10. Big Picture Summary

Let's bring everything together with a high-level view of the landscape of word embedding methods.

Method	Core Idea	Strengths	Weaknesses
One-Hot	Assign unique index to each word	Simple; no training needed	No meaning; huge dimensionality
Co-Occurrence Counts	Count how often words appear together	Interpretable; easy to implement	Sparse; frequency-dominated
PPMI	Use mutual information instead of counts	State-of-art pre-neural; better than raw counts	Still sparse; requires SVD for compression
LSA / SVD	Reduce count matrix with SVD	Also gives document vectors; topic modeling	Misses syntactic structure
Word2Vec	Train neural model to predict context	Fast; high quality; linear analogies	Context-independent (one vector per word)
GloVe	Combine global stats + learned objective	Uses all corpus data; good analogies	Slightly weaker than W2V with tuning
FastText	Add character n-gram vectors	Handles OOV; robust to morphology & typos	Slower; larger model size

The Evolution of the Field

The field progressed through three phases:

1. Sparse, hand-crafted representations (one-hot, count matrices) — interpretable but don't capture meaning
2. Dense, learned static embeddings (Word2Vec, GloVe, FastText) — compact, meaningful, but one vector per word regardless of context
3. Contextual embeddings (ELMo, BERT, GPT) — the word "bank" gets different vectors in "river bank" vs. "savings bank" — this is the current state of the art, covered in later course material

> **Ỗ Key Concept: Why Word Embeddings Still Matter**
>
> Even though contextual models (BERT, GPT) have superseded static embeddings for many tasks, the core ideas remain essential. Distributional semantics, co-occurrence statistics, negative sampling, and the geometry of semantic spaces are all foundational concepts that appear throughout modern NLP. Understanding Word2Vec deeply makes it much easier to understand transformers and large language models.

— End of Word Embeddings Guide —